**UGANDA TAX ENGINE**

The Uganda Tax Engine is a light weight, high precision Python module designed to calculate statutory payroll taxes (PAYE) and Value Added Tax (VAT) in full compliance of with Uganda Revenue Authority.

**Why is float unsafe for tax computation. Show concrete failing example**

Standard binary IEEE 754 binary floating-point representation stores numbers in binary (base 2). Financial systems operating in decimal (base-10) require representation of decimal fractions like 0.1 or 0.18 (VAT). In binary floating-point, these numbers become repeating binary fractions that cannot be represented with perfect exactness.

In large scale payroll or multi-transaction systems, these tiny imprecisions compound into significant financial discrepancies, audit failures and rounding errors

**Concrete Failing example**

#Trailing precision artifacts in financial math

gross_amount=100000.15

vat rate=0.18

bad_vat=gross* vat_rate

print(bad_vat)

# Output: 18000.027000000002 ……unwanted trailing binary garbage

# Accumulation errors across multiple payroll records

total_tax =0.0

for _ in range (10):

    total_tax += 0.1

print(total_tax)

# Output: 0.999999999999999 ……Expected exactly 0.1

**Solution in Tax Systems**

To prevent arithmetic drift:

Use python built-in decimal.Decimal module for fixed-point decimal arithmetic.

Apply explicit monetary rounding (round(value,2)) at key statutory boundaries

Where would inheritance be justified here, and where did you deliberately avoid it.

**Inheritance justified**

Multi-jurisdictional Tax Engines: If extending the engine across East African Community member states, an abstract base class TaxEngine could define a unified interface (calculate_paye (), calculate_vat ()).

Concrete substances such as UgandaTaxEngine, KenyaTaxEngine, and RwandaTaxEngine would implement country-specific tax rules.

Taxpayer Classification Hierarchies: Modelling distinct entity types (e.g base class Taxpayer with specialized subclasses like ResidentIndividual, NonResidentIndividual, or CorporateEntity) would allow sharing common payroll attributes while enforcing custom exemption rules.

**Inheritance deliberately avoided**

Stateless Statutory logic: The core algorithms for PAYE and flat VAT percentages are pure mathematical functions. Forcing them into complex inheritance hierarchies would introduce unnecessary overhead, tight coupling, and the “fragile base class” problem.

Modular Testability: Using static method(@staticmethod) or standalone utility functions keeps the codebase decoupled, lightweight, and straightforward to unit test using pytest without managing class state or inheritance side-effects.

**Findings**

Progressive statutory compliance

PAYE follows URA’s progressive bracket system: income up to 235000 ugx is tax exempt, with incremental tiers at 10% 20% and 30%

High earners (>10000000 ugx gross) are subject to additional 10% surtax on chargeable income above 10M ugx

Value and edge case protection;

Guard clauses prevent processing negative income values by throwing explicit ValueError exceptions

Calculations adhere to exact monetary rounding to protect against floating-point accumulation drift.

Automated verification;

Comprehensive pytest test suites verify bracket, tax exemptions and boundary threshold limits.

**Academic Integrity and AI use declaration**

In accordance with course guidelines, AI coding assistants (specifically Gemini) were utilized during the development of this project collaborative tools for:

structuring project layout and refining technical documentation.

Generating unit test boilerplate templates and verifying edge-case coverage.

Guidance on Git command workflows and repository management

All underlying business logic, calculation verifications, code implementation and test executions were fully reviewed executed and validated by the author